

Relatório — Experimentos de TCP e UDP no monitor Linux

Aluno: Samuel Morales e Arthur Avila
Curso: Ciência da Computação
Disciplina: Laboratório de Redes de Computadores
Data: 28/09/2026

1. Formato dos cabeçalhos TCP e UDP

1.1 Comparação dos campos

Campo	UDP	TCP	Função
Porta de origem	Presente ▾	Presente ▾	Identifica a aplicação/processo de origem.
Porta de destino	Presente ▾	Presente ▾	Identifica a aplicação/processo de destino.
Comprimento (Length)	Presente ▾	Ausente ▾	No UDP informa o tamanho do datagrama UDP.
Checksum	Presente ▾	Presente ▾	Detecção de erros.
Número de sequência	Ausente ▾	Presente ▾	Identifica a posição dos bytes no fluxo TCP.
Número de ACK	Ausente ▾	Presente ▾	Indica o próximo byte esperado.
Tamanho do cabeçalho	8 bytes ▾	20 bytes no mínimo ▾	Define o início dos dados.
Flags de controle	Ausente ▾	Presente ▾	Controlam a conexão, como SYN, ACK e FIN.
Janela (Window)	Ausente ▾	Presente ▾	Controle de fluxo.
Ponteiro de urgente	Ausente ▾	Presente ▾	Permite indicar dados urgentes.
Opções	Ausente ▾	Presente/opcional ▾	Recursos como MSS, SACK e Window Scale.

O roteiro solicita essa comparação dos campos presentes nos dois cabeçalhos.

1.2 Tamanho dos cabeçalhos e sobrecarga

O cabeçalho UDP possui 8 bytes, enquanto o cabeçalho TCP possui 20 bytes no formato mínimo.

Na captura DNS, o datagrama UDP tinha 73 bytes, portanto a sobrecarga do cabeçalho UDP foi:

$$8 / 73 * 100 \approx 10,96\%$$

Para o TCP, considerando a requisição HTTP do frame 8, havia 92 bytes de dados TCP e um cabeçalho TCP mínimo de 20 bytes:

$$20 / (20 + 92) * 100 \approx 17,86\%$$

Isso mostra que, para mensagens pequenas, o TCP possui maior overhead relativo devido aos mecanismos adicionais de controle e gerenciamento da conexão. O roteiro solicita justamente a comparação da sobrecarga dos cabeçalhos.

1.3 Campo Length do UDP e TCP

O UDP possui um campo Length porque cada datagrama UDP possui seu próprio tamanho.

O TCP não necessita de um campo Length equivalente porque o tamanho dos dados TCP pode ser obtido a partir do tamanho total informado pelo protocolo IP, descontando os tamanhos dos cabeçalhos IP e TCP.

1.4 Checksum offloading

O Wireshark pode apresentar o checksum como não verificado ou incorreto em pacotes enviados pelo próprio computador devido ao checksum offloading.

Nesse mecanismo, o cálculo final do checksum pode ser realizado pela interface de rede depois que o pacote já foi observado pelo mecanismo de captura. Assim, o Wireshark pode visualizar temporariamente um valor que ainda não representa o checksum final.

2. UDP sem conexão e sem estado

2.1 Quantidade de datagramas

Uma consulta DNS simples realizada através de UDP normalmente envolve:

- 1 datagrama de consulta;
- 1 datagrama de resposta.

Portanto, são 2 datagramas.

O TCP precisaria de segmentos adicionais para estabelecer e encerrar a conexão, além dos segmentos contendo os dados. Por isso, o UDP é adequado para consultas DNS pequenas, pois evita o custo de estabelecer uma conexão.

O roteiro destaca essa comparação entre DNS/UDP e TCP.

2.2 Porta de origem

Na captura realizada, a consulta DNS para `www.pucrs.br` utilizou:

`10.53.0.10:51119 → 8.8.8.8:53`

Portanto:

- **Porta de origem:** 51119
- **Porta de destino:** 53

A porta de origem é escolhida dinamicamente pelo sistema operacional. O uso de portas de origem diferentes permite distinguir consultas simultâneas e também faz parte das técnicas de randomização de portas utilizadas pelo DNS.

Observação: a captura que preservamos não contém as três consultas A, AAAA e NXDOMAIN simultaneamente, então não é correto inventar os valores das outras duas portas.

2.3 Como o cliente identifica a resposta?

O UDP não possui número de sequência e não estabelece uma conexão.

O DNS resolve essa questão utilizando o campo: **Transaction ID**

Na captura, por exemplo:

Transaction ID: 0x8457

A resposta DNS possui o mesmo identificador, permitindo associá-la à consulta correspondente.

2.4 DNSKEY, truncamento e TCP

Foi executado:

`dig +dnssec +bufsize=512 org DNSKEY @8.8.8.8` A captura mostrou:

- Consulta DNSKEY via UDP;
- Resposta UDP com TC = 1 (Truncated);
- Estabelecimento de conexão TCP;
- Repetição da consulta DNSKEY via TCP;
- Resposta DNSKEY através de TCP com 1375 bytes.

A flag TC indica que a mensagem DNS foi truncada. Como a resposta não pôde ser entregue integralmente através do UDP, o cliente repetiu a consulta utilizando TCP.

O EDNS permite aumentar o tamanho das mensagens DNS transportadas por UDP, portanto o antigo limite de 512 bytes não deve ser confundido com o tamanho máximo de um datagrama UDP. O próprio roteiro solicita essa relação entre TC, EDNS e TCP.

2.5 Datagrama enviado para a porta 6666

Foi enviado:

10.53.0.10:42567 → 8.8.8.8:6666 com 5 bytes de dados.

A captura mostrou apenas o datagrama UDP enviado e não apresentou uma resposta ICMP.

Portanto, não é possível afirmar que um ICMP específico foi capturado nesse experimento.

Em uma situação em que uma porta UDP fechada respondesse explicitamente, seria esperado:

- ICMP Destination Unreachable
- Type 3
- Code 3 — Port Unreachable

Já um destino filtrado pode simplesmente não responder, não permitindo distinguir a situação apenas pela ausência de resposta.

3. Conexão, sequência e confirmação no TCP

3.1 Handshake

Na captura HTTP, o handshake observado foi:

SYN

10.53.0.10:36626 → 90.130.70.73:80

Seq=0

Flags=SYN

SYN/ACK

90.130.70.73:80 → 10.53.0.10:36626

Seq=0

Ack=1

Flags=SYN, ACK

ACK

10.53.0.10:36626 → 90.130.70.73:80

Seq=1

Ack=1

Flags=ACK

Portanto:Cliente

Servidor

SYN ----->

<----- SYN/ACK

ACK ----->

ISN

O Wireshark está configurado para apresentar números de sequência relativos, por isso a captura mostra Seq=0 nos SYNs. Os valores reais dos ISNs não foram registrados na nossa análise.

Os ISNs reais são escolhidos pelo TCP e não precisam começar em zero; o Wireshark utiliza numeração relativa para facilitar a leitura da conexão. A documentação do Wireshark também descreve o uso de números de sequência e a análise do próximo número esperado.

3.2 Sequenciamento

Foi utilizada a conexão:

90.130.70.73:80 → 10.53.0.10:43222

Cinco segmentos consecutivos observados foram:

Quadro	Seq	Len	Próximo Seq
57	1	8568	8569
62	8569	5712	14281
71	14281	2856	17137
80	17137	2856	19993
89	19993	2856	22849

Verificação:

- $1 + 8568 = 8569$
- $8569 + 5712 = 14281$
- $14281 + 2856 = 17137$
- $17137 + 2856 = 19993$
- $19993 + 2856 = 22849$

Portanto:

$$\text{Seq}(n+1) = \text{Seq}(n) + \text{Len}(n)$$

O número de sequência TCP representa a posição dos bytes dentro do fluxo, e não simplesmente o número do segmento. O Wireshark também utiliza essa relação para calcular o próximo número de sequência esperado.

3.3 Confirmação cumulativa

Na captura HTTP, um exemplo é:

Seq=1
Len=92

seguido por um ACK:

Ack=93

O ACK=93 indica que os bytes até o número 92 foram recebidos e que o próximo byte esperado é o 93.

O TCP utiliza confirmações cumulativas e não necessariamente envia um ACK individual para cada segmento. O mecanismo de delayed ACK permite aguardar um pequeno intervalo antes de enviar uma confirmação, dependendo das condições da comunicação.

3.4 Segmentação, MSS e MTU

No handshake TCP foram observados:

- MSS do cliente = 1460 bytes
- MSS do servidor = 65480 bytes

A interface eth0 apresentou:

- MTU = 1500 bytes

Para IPv4, considerando os cabeçalhos IP e TCP mínimos:

$$1500 - 20 - 20 = 1460$$

Portanto, o MSS de 1460 bytes é compatível com uma MTU de 1500 bytes.

Na transferência foram observados segmentos TCP de dados com tamanhos como 2856 bytes no campo Len exibido pelo Wireshark, devido à forma como a captura e a reassemblagem são apresentadas. O MSS anunciado continua sendo 1460 bytes; o Wireshark identifica alguns pacotes como segmentos de uma PDU TCP reassemblada.

3.5 Ordenação e reconstrução do fluxo

O Wireshark pode utilizar Follow → TCP Stream para reconstruir o fluxo HTTP.

Isso é possível porque cada segmento possui um número de sequência, que permite determinar a posição dos bytes dentro do fluxo.

O mecanismo de reassemblagem TCP do Wireshark coleta os segmentos e os entrega ao protocolo de camada superior, como HTTP, na ordem correta.

3.6 Encerramento

Na captura HTTP foram observados os segmentos:

10.53.0.10:36626 → 90.130.70.73:80
FIN, ACK

seguido por:

90.130.70.73:80 → 10.53.0.10:36626
FIN, ACK

e posteriormente o ACK final.

Isso demonstra o encerramento ordenado da conexão TCP.

Uma consulta DNS simples utilizou poucos datagramas UDP e não necessita de handshake ou encerramento de conexão. Já uma comunicação TCP possui handshake, dados, confirmações e encerramento. Portanto, o TCP apresenta maior overhead de controle.

4. Multiplexação e demultiplexação

4.1 Conexões simultâneas

Foram observadas 3 conexões TCP simultâneas para o mesmo servidor (90.130.70.73:80):

Conexão	Origem	Destino	Pacotes	Dados
1	10.53.0.10:43194	90.130.70.73:80	1412	811 kB
2	10.53.0.10:43208	90.130.70.73:80	1443	812 kB
3	10.53.0.10:43222	90.130.70.73:80	1381	782 kB

Essas informações foram obtidas em Statistics → Conversations → TCP.

4.2 O que muda entre as conexões?

O IP de destino e a porta de destino são iguais:
90.130.70.73:80

O que muda é principalmente a porta de origem:

- 43194
- 43208
- 43222

Assim, cada conexão possui uma 4-tupla distinta.

4.3 Associação com os processos

O comando `ss -tnp` mostrou:

- PID 418 → 10.53.0.10:43194
- PID 419 → 10.53.0.10:43208
- PID 420 → 10.53.0.10:43222

Os três processos eram `curl`.

A estrutura utilizada pelo sistema operacional para representar cada conexão é o socket TCP, identificado pelo conjunto de endereços e portas da comunicação.

O roteiro solicita justamente relacionar as conexões observadas no Wireshark às linhas do `ss -tnp`.

4.4 Independência das três conexões

As três conexões utilizaram o mesmo servidor, mas foram mantidas independentemente devido às portas de origem diferentes.

Os dados observados foram aproximadamente:

- **Conexão 43194:** 811 kB
- **Conexão 43208:** 812 kB
- **Conexão 43222:** 782 kB

As durações observadas foram aproximadamente:

- **43194:** 9,9748 s
- **43208:** 9,6358 s
- **43222:** 9,9390 s

Assim, o servidor consegue manter três fluxos independentes mesmo utilizando o mesmo endereço IP e a mesma porta de destino.

5. Conclusão

Os experimentos demonstraram as principais diferenças entre UDP e TCP.

O UDP apresenta estrutura simples e não estabelece conexão, sendo adequado para consultas DNS pequenas. O TCP, por outro lado, estabelece uma conexão, utiliza números de sequência, confirmações, controle de fluxo e mecanismos de encerramento, oferecendo um fluxo ordenado e confiável.

As capturas também demonstraram o funcionamento do DNS sobre UDP e o fallback para TCP quando uma resposta DNS é truncada. Na comunicação HTTP, foi possível observar o handshake, o sequenciamento dos bytes, as confirmações e o encerramento da conexão.

Por fim, as três conexões HTTP simultâneas mostraram o funcionamento da multiplexação e demultiplexação, utilizando diferentes portas de origem para distinguir conexões que possuem o mesmo servidor e porta de destino.